

PRECIS

CTO Technical Brief

From Prototype to MVP/Beta

What exists, what you need to build, and how it all fits together.

Confidential — March 2026

Prepared for CTO review. Contains full technical architecture, repository structure, and phased implementation plan.

1. What Already Exists

The repository you are receiving contains 65 files representing a complete visual prototype with several production-ready subsystems. This is not a wireframe or a design mockup — every page is a functional React component with real interactivity, theme switching, and data-driven rendering.

Repository Structure

```
precis-deploy/
■■ src/components/ — 26 React page builds (JSX)
■■ src/lib/ — Shared state store + Synopsis Engine module
■■ src/security/ — AuthProvider, sanitization, secureApiClient
■■ server/ — Express API proxy (Synopsis Engine backend)
■■ public/ — Favicon, OG images
■■ extension/ — Chrome extension (Compass v3.0)
■■ docs/ — 9 legal documents, audit report, integration guide
■■ Config: package.json, vite.config.js, render.yaml, index.html
```

What Is Production-Ready Now

Component	Status	Notes
Express API proxy (server/index.js)	READY	Rate limiting, CORS, input validation, Copyright Guard v1.2
Synopsis Engine (precis-engine-shared.js)	READY	6 lens prompts, 3 formats, fiction/nonfiction toggle, cost estimation
Auth screen (precis-auth.jsx)	READY*	Apple/Google/Email UI. *Mock auth; Clerk integration points in comments
Library Import (precis-import.jsx)	READY	Goodreads + StoryGraph CSV parser, auto-detection, preview
App Shell (precis-app-shell.jsx)	READY*	Auth → Onboarding → Import → Feed routing. *localStorage, not DB
Security: AuthProvider.jsx	READY*	RBAC, IDOR protection, CSRF. *Mock mode; production Clerk swap noted
Security: sanitization.js	READY	XSS prevention, input sanitization
Security: secureApiClient.js	READY	CSRF tokens, rate limiting, encrypted headers
24 UI page builds	VISUAL	Fully interactive prototypes with mock data. Need DB wiring.
9 legal documents (.docx)	DRAFT	TOS, Privacy, IP License, etc. Awaiting attorney review.
Chrome extension	READY	Compass v3.0. Separate deploy to Chrome Web Store.

READY = deploy as-is. READY = functional but needs production swap (comments show exactly what to change). VISUAL = UI complete, needs backend wiring.*

2. What Needs to Be Built

The gap between prototype and MVP is concentrated in three areas: a persistent data layer (database), production authentication (Clerk activation), and API wiring to replace mock data with real queries.

2.1 Database Layer (PostgreSQL + Prisma)

The most significant piece of work. Every UI page currently renders mock data from hardcoded arrays. The schema design is partially modeled in the existing builds. A schema.prisma file exists in the project repo as a starting point.

Core Tables Required:

Table	Key Fields	Priority
users	id, clerkId, email, name, handle, lens, identity, ink, createdAt	P0
posts	id, authorId, type (original/review/rec/spoiler), title, body, bookId, rating, cw, publishedAt	P0
books	id, title, author, isbn, coverUrl, pageCount, genres, openLibraryId	P0
follows	followerId, followingId, createdAt	P0
likes	userId, postId, createdAt	P0
shelves	userId, postId, createdAt (shelving a post)	P0
user_books	userId, bookId, status (read/reading/want), rating, progress, startedAt, finishedAt	P0
ink_events	userId, amount, reason (post/like_received/milestone/etc), createdAt	P1
notifications	id, recipientId, type, actorId, postId, read, createdAt	P1
messages	id, conversationId, senderId, body, createdAt, readAt	P1
conversations	id, participant1Id, participant2Id, lastMessageAt	P1
book_clubs	id, name, description, bookId, creatorId, isPublic, createdAt	P2
club_members	clubId, userId, role, joinedAt	P2
waypoints	id, userId, bookId, pageNumber, body, createdAt	P2
challenges	id, title, description, type, startDate, endDate, creatorId	P2
imports	id, userId, source (goodreads/storygraph), bookCount, importedAt	P1

P0 = required for beta launch. P1 = needed within first month. P2 = can ship incrementally.

2.2 Clerk Authentication (Production Activation)

Clerk account is already created. Publishable key is set. The auth screen (precis-auth.jsx) has Apple/Google/Email buttons with Clerk integration code written in comments at every hook point.

Actionable steps:

- In Clerk dashboard: Enable Apple Sign In, Google OAuth, and Email/Password providers
- For Apple: Register an App ID and Service ID in Apple Developer Console, add credentials to Clerk
- For Google: Create OAuth 2.0 credentials in Google Cloud Console, add to Clerk

- In Clerk dashboard: Set allowed origins to your Render domain (e.g. <https://precis-platform.onrender.com>)
- In `precis-auth.jsx`: Uncomment the Clerk imports and replace mock handlers with real `signIn/signUp` calls
- In `main.jsx`: Wrap the app in `ClerkProvider` with the publishable key
- Store Clerk `userId` as `clerkId` in the `users` table to link auth identity to platform profile
- Use Clerk's `unsafeMetadata` to persist `lens/handle` during onboarding before DB is ready

2.3 API Routes (Express Expansion)

The Express server currently handles only Synopsis Engine routes (`/api/synopsis`, `/api/qa`, `/api/health`). It needs to be expanded to serve platform data.

New routes needed:

- GET `/api/feed` — paginated feed (following + algorithm mix)
- GET `/api/user/:handle` — public profile
- POST `/api/posts` — create post (with type, bookRef, content warning)
- POST `/api/posts/:id/like`, `/api/posts/:id/shelf` — engagement
- GET `/api/books/search?q=` — book search (OpenLibrary API proxy)
- POST `/api/books/import` — process Goodreads/StoryGraph CSV (server-side validation)
- GET `/api/notifications` — notification feed
- GET `/api/messages`, POST `/api/messages` — DM system
- POST `/api/auth/webhook` — Clerk webhook to sync user creation to DB

3. Phased Implementation Plan

Phase 1: Foundation (Weeks 1–2)

Goal: Users can create real accounts, complete onboarding, and see a feed.

- **Set up PostgreSQL on Supabase or Render.** Create the P0 tables (users, posts, books, follows, likes, shelves, user_books). Prisma ORM recommended — a schema.prisma starter exists in the repo.
- **Activate Clerk production auth.** Uncomment Clerk imports in precis-auth.jsx, wrap app in ClerkProvider, set up Clerk webhook to POST /api/auth/webhook on user creation (syncs clerkId → users table).
- **Wire onboarding to DB.** When user completes onboarding, write profile data (name, handle, lens, identity, books, follows) to users and user_books tables. Replace localStorage persistence with API calls.
- **Wire feed to DB.** Replace mock FEED_ITEMS array in precis-feed-v8.jsx with fetch('/api/feed') call. Server queries posts from followed users + algorithmic surfacing.
- **Wire library import to DB.** When user confirms CSV import, POST /api/books/import sends parsed books to server. Server matches against OpenLibrary, creates book records, and populates user_books.
- **Deploy to Render.** Set env vars (ANTHROPIC_API_KEY, VITE_CLERK_PUBLISHABLE_KEY, DATABASE_URL). Verify health check at /api/health.

Phase 2: Core Social (Weeks 3–4)

Goal: Users can post, like, shelf, follow, and see each other's profiles.

- **Wire compose to DB.** POST /api/posts creates a real post. Support all 4 types: original work, review, recommendation, spoiler zone. Include content rating (G/T/M) per TOS.
- **Wire profile to DB.** GET /api/user/:handle returns real user data, posts, reading maps, Ink score. Profile page already renders all of this from props.
- **Implement Ink accrual.** Insert ink_events on: post published (+5), like received (+1), shelf received (+2), first post milestone (+10). Sum events for display. Tier calculation already exists in every build.
- **Wire notifications.** On like/shelf/follow/comment, insert into notifications table. GET /api/notifications returns unread. Push via SSE or polling.
- **Wire explore/search.** GET /api/explore returns trending books, active readers, lens-matched content. Book search proxies to OpenLibrary API.

Phase 3: Communication + Community (Weeks 5–6)

Goal: Users can DM, join book clubs, set waypoints, and participate in challenges.

- **Wire messages.** Conversation-based DM system. The messages UI (precis-messages.jsx) is fully built with typing indicators, shared book context, and Ink badges.

- **Wire book clubs.** Create, join, and manage clubs. Discussion threads per chapter. The UI (precis-book-clubs.jsx) handles all of this visually.
- **Implement waypoints.** Reading reflections tied to specific page numbers. Enables spoiler-safe discussion at arbitrary points in a book. Unique differentiator.
- **Wire challenges.** Monthly reading goals. Creator + community challenges. Progress tracking. UI is complete.
- **Implement cross-book spoiler detection.** The feed already flags crossBookSpoiler on posts. Backend needs to check if a review of Book A spoils Book B.

4. Key Technical Decisions Already Made

Decision	Choice	Rationale
Frontend framework	React (Vite)	All 26 builds are React. No migration needed.
Backend	Express (Node.js)	Server already running. Expand, don't replace.
Auth	Clerk	Apple/Google/Email out of the box. AuthProvider built. Free tier = 10K MAU.
Database	PostgreSQL	Schema.prisma exists. Prisma ORM. Supabase or Render Postgres.
Hosting	Render	render.yaml configured. Auto-deploy from GitHub. SSL included.
API design	REST	Synopsis Engine already REST. Extend pattern for platform routes.
Payments	Stripe	Webhook verification module exists (stripeWebhook.js). Not yet wired.
AI model	Claude Sonnet 4.6	Synopsis Engine uses this. Copyright Guard v1.2 prevents reproduction.
Themes	4-theme system	Twilight, Parchment, Stone, Editorial. All builds support switching.
Reputation	Ink (5 tiers)	Fresh Ink → Indelible. Standardized across all 26 builds.
Security	RBAC + IDOR + CSRF + AES-256-GCM	256-bit security module. 3 client-ready, 7 await Express.

5. Entity Structure (Critical for Legal Compliance)

Two-entity architecture. Get this wrong and the corporate veil is compromised.

Context	Entity	Why
TOS / Privacy Policy	Precis Social, LLC	Users contract with the operating company
Copyright footers	Precis Technologies, Inc.	IP holder owns the copyright
IP ownership	Precis Technologies, Inc.	Holding co owns all IP, licenses to opco
Contractor agreements	Precis Social, LLC	Opco is the contracting party
Domains / Trademarks	Precis Technologies, Inc.	IP holder registers marks
Stripe / payment processing	Precis Social, LLC	Opco runs operations
Hosting / vendor contracts	Precis Social, LLC	All operational SaaS through opco

6. Required Skillset

The CTO role for this phase requires a fullstack developer who can work independently across the following stack:

Must-Have Technical Skills:

- **React (functional components, hooks, context, lazy loading)** — all 26 builds use this. No class components. Heavy use of `useState`, `useEffect`, `useRef`, `useCallback`.
- **Node.js + Express** — server already running. You will add 15+ API routes to it.
- **PostgreSQL + Prisma ORM** — schema design, migrations, efficient queries for feeds (paginated, sorted by relevance + time), full-text search for books/users.
- **Clerk (or equivalent auth provider)** — OAuth flows, webhook handling, session management, JWT verification. Clerk-specific: custom metadata, user sync.
- **REST API design** — authentication middleware, input validation, error handling, rate limiting. The security module provides patterns for all of these.
- **CSV parsing / data normalization** — the import component parses client-side, but server-side validation and book matching (OpenLibrary API) is needed.
- **Deployment: Render (or similar PaaS)** — environment variables, build commands, health checks, logs, custom domains, SSL.

Strong-to-Have:

- **Stripe integration** — for future premium subscriptions and Paper stored value. Webhook module already exists.
- **WebSocket or SSE** — for real-time notifications and DM typing indicators.
- **OpenLibrary / Google Books API** — book metadata enrichment for imports and search.
- **Anthropic API** — the Synopsis Engine is already built, but expanding AI features (AI-assisted content moderation, Taste Graph signals) will require prompt engineering.
- **Chrome Extension development** — Compass v3.0 is built and functional, but future versions will need maintenance.

Non-Technical but Critical:

- **Respect the brand identity.** The literary voice, the theme system, the Ink metaphor, the lens framework — these are not decorations. They are the product. Don't flatten them to ship faster.
- **Security-first mindset.** The security module is built premier from the ground up. Integrate it, don't bypass it. RBAC checks on every protected route. Input sanitization on every user input. IDOR protection on every resource access.
- **Entity awareness.** Every vendor contract, every API key, every SaaS subscription goes under Precis Social, LLC (opco). Not Technologies (holding co). Not personal accounts.

7. First Week: Day-by-Day Checklist

Day 1: Orientation

- Clone the repository. Run `npm install` & `npm run dev`. Click through every page.
- Read the ecosystem audit document (`docs/precis_ecosystem_audit.docx`) for context on design decisions.
- Read the integration guide (`docs/precis-integration-guide.md`) for wiring patterns.
- Read `COPYRIGHT_GUARDRAILS.md` to understand the AI content generation constraints.

Day 2: Database

- Provision PostgreSQL (Supabase free tier or Render Postgres).
- Initialize Prisma: `npx prisma init`. Create P0 schema (users, posts, books, follows, likes, shelves, user_books).
- Run first migration: `npx prisma migrate dev`.
- Verify connection from Express server. Add `DATABASE_URL` to `.env.local` and Render.

Day 3: Authentication

- Activate Clerk providers (Apple, Google, Email) in the Clerk dashboard.
- Set `VITE_CLERK_PUBLISHABLE_KEY` in Render. Set Clerk allowed origins.
- Uncomment Clerk imports in `precis-auth.jsx`. Replace mock handlers.
- Wrap app in ClerkProvider in `main.jsx`. Test sign-up and sign-in flows.
- Set up Clerk webhook: `POST /api/auth/webhook` creates user record in DB on signup.

Day 4: Wire Onboarding + Import

- On onboarding completion: `POST /api/users/onboard` writes profile to DB.
- On library import: `POST /api/books/import` processes CSV server-side, creates book + user_books records.
- Match imported books against OpenLibrary API for enriched metadata.
- Test the full flow: sign up → onboard → import → feed.

Day 5: Wire Feed

- Implement `GET /api/feed` (paginated, returns posts from followed users + algorithm mix).
- Replace mock `FEED_ITEMS` in `precis-feed-v8.jsx` with real API call.
- Implement `POST /api/posts` for the compose flow.
- Test: sign up, import, write a post, see it appear in feed.
- Deploy to Render. Verify health check. Share the URL.

8. Environment Variables (Render Dashboard)

Variable	Where to Get It	Required
ANTHROPIC_API_KEY	console.anthropic.com → API Keys	Yes
VITE_CLERK_PUBLISHABLE_KEY	clerk.com → Dashboard → API Keys	Yes
DATABASE_URL	Supabase or Render Postgres connection string	Yes
RATE_LIMIT_PER_MIN	Default: 6. Set higher for testing.	No
NODE_ENV	production	Auto-set by Render
PORT	Auto-set by Render	Auto

CRITICAL: Never commit API keys to the repository. They go in Render's dashboard only. The `.env.example` file shows what's needed without real values.